



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



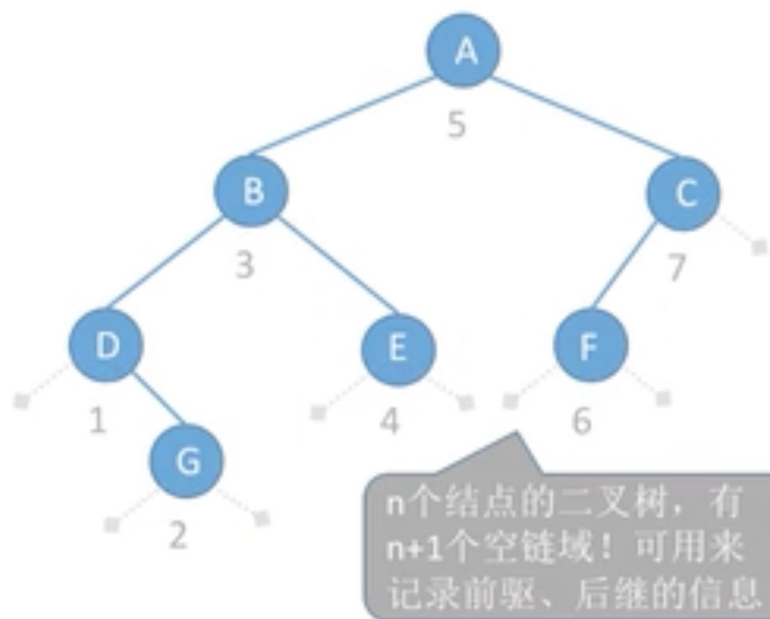
第五章 树与二叉树



- 树和森林的概念
- 二叉树
- 二叉树遍历
- 线索化二叉树
- 树与森林
- 堆
- Huffman树

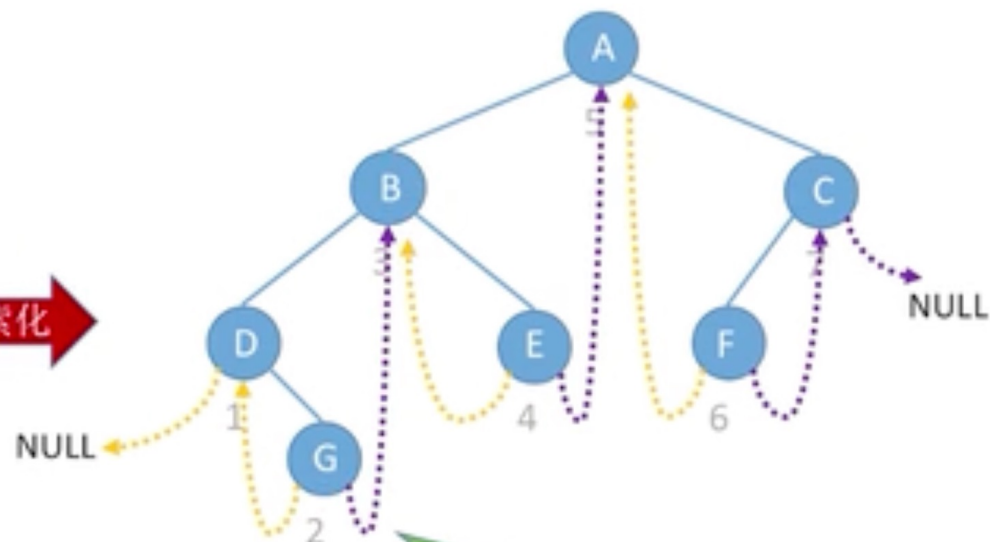


示例



n个结点的二叉树，有n+1个空链域！可用来记录前驱、后继的信息

中序遍历序列：DGBEAFC



问题：如何找到G的后继？

图示说明

前驱线索（由左孩子指针充当）：

后继线索（由右孩子指针充当）：

指向前驱、后继的指针称为“线索”



- 线索化二叉树的主要目的是提高遍历效率和节省存储空间。
- 提高遍历效率
 - 普通遍历需要栈或递归
 - 线索化后：可以直接找到前驱和后继节点；无需栈辅助， $O(1)$ 空间复杂度遍历
- 解决空指针浪费问题
 - 普通二叉树中： n 个节点的二叉树有 $2n$ 个指针域；实际使用的指针： $n-1$ 个；空指针数量： $n+1$ 个（浪费约50%）
 - 线索化利用这些空指针存储遍历信息。



5.5 线索化二叉树 (Threaded Binary Tree)

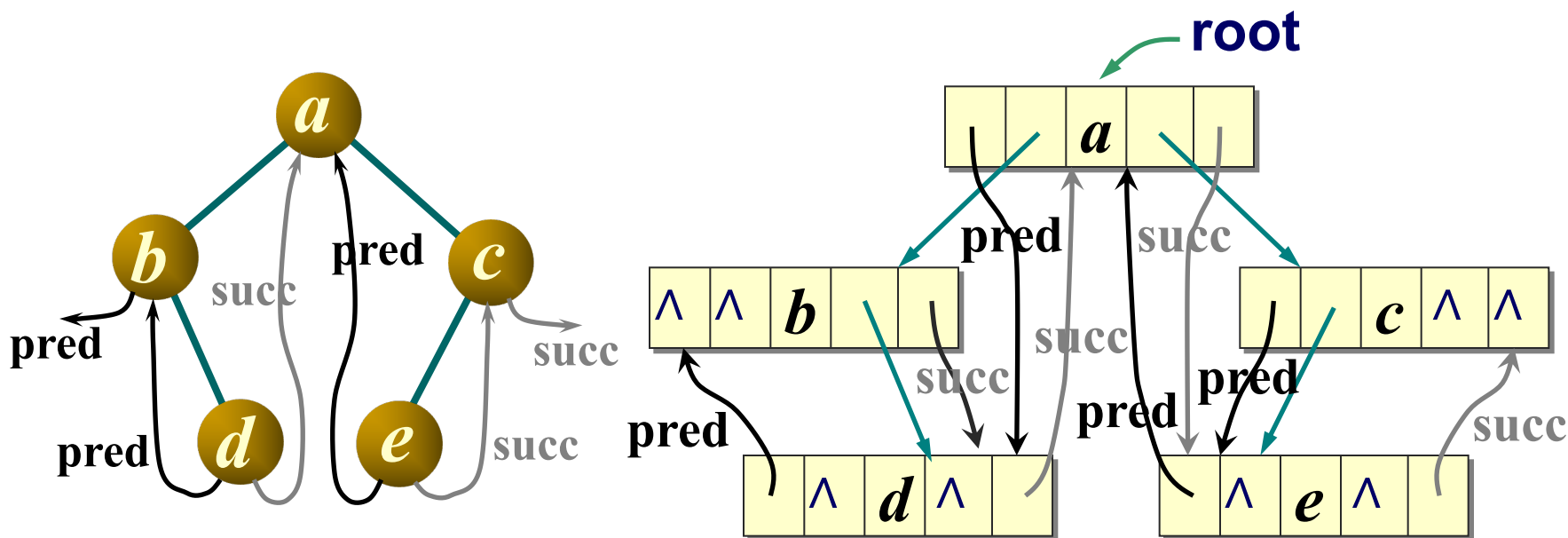
- 又称为穿线树。
- 通过二叉树的遍历，可将二叉树中所有结点的数据排列在一个线性序列中，可以找到某数据在这种排列下它的**前驱**和**后继**。
- 希望不必每次都通过遍历找出这样的线性序列。只要事先做预处理，**将某种遍历顺序下的前驱、后继关系记在树的存储结构中**，以后就可以高效地找出某结点的前驱、后继。



5.5.1 线索 (Thread)



pred	leftChild	data	rightChild	succ
------	-----------	------	------------	------



中序遍历序列: bdaec

方法一：增加 Pred 指针和 Succ 指针的二叉树



- 这种设计的缺点是每个结点增加两个指针，当结点数很多时，存储消耗较大。
- 对于原来的二叉链表结构，一棵 n 个结点的二叉树共有 $2n$ 个指针域，而非空的指针域为 $n-1$ 个，因此，仍有 $n+1$ 个指针域没有利用起来。



方法二：增加 左右线索标志 的二叉树

- 改造树结点，将 pred 指针和 succ 指针压缩到 leftChild 和 rightChild 的空闲指针中，并增设两个标志 ltag 和 rtag，指明指针是指示子女还是前驱 / 后继。后者称为线索。

leftChild	ltag	data	rtag	rightChild
-----------	------	------	------	------------

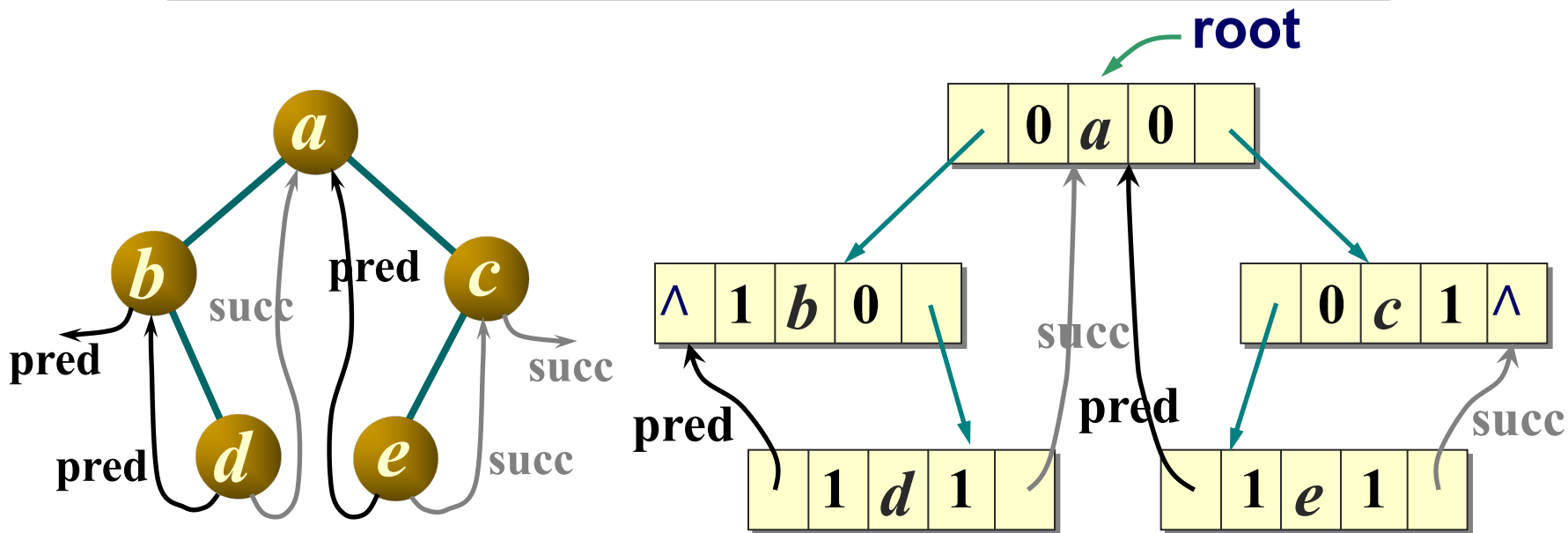
- 当 ltag (或 rtag) = 0，表示相应指针指示左子女 (或右子女结点)；
- 当 ltag (或 rtag) = 1，表示相应指针为前驱 (或后继) 线索。



线索化二叉树及其链表表示



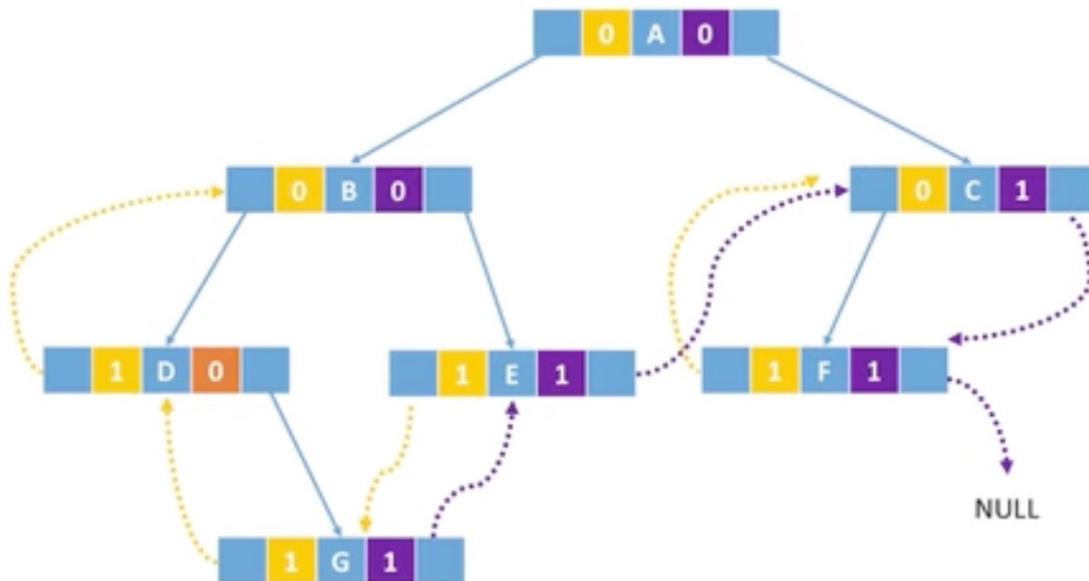
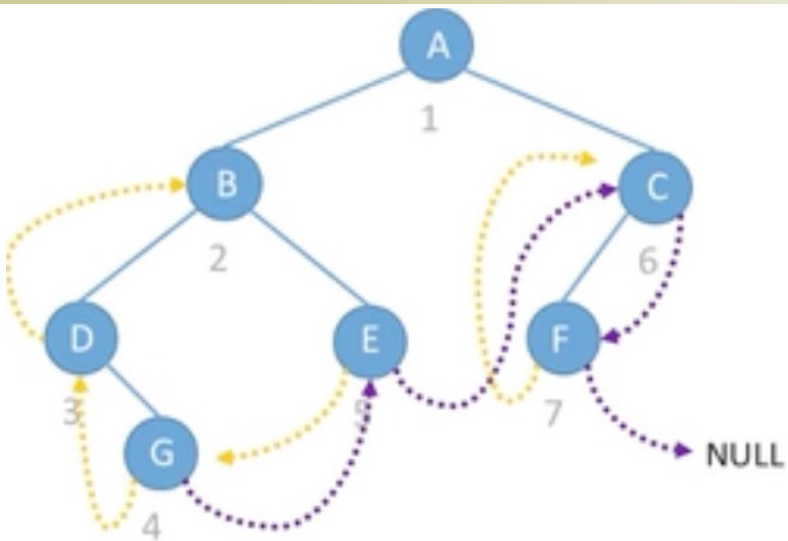
leftChild	ltag	data	rtag	rightChild
-----------	------	------	------	------------



ltag = 0, *leftChild* 为左子女指针
ltag = 1, *leftChild* 为前驱线索
rtag = 0, *rightChild* 为右子女指针
rtag = 1, *rightChild* 为后继线索



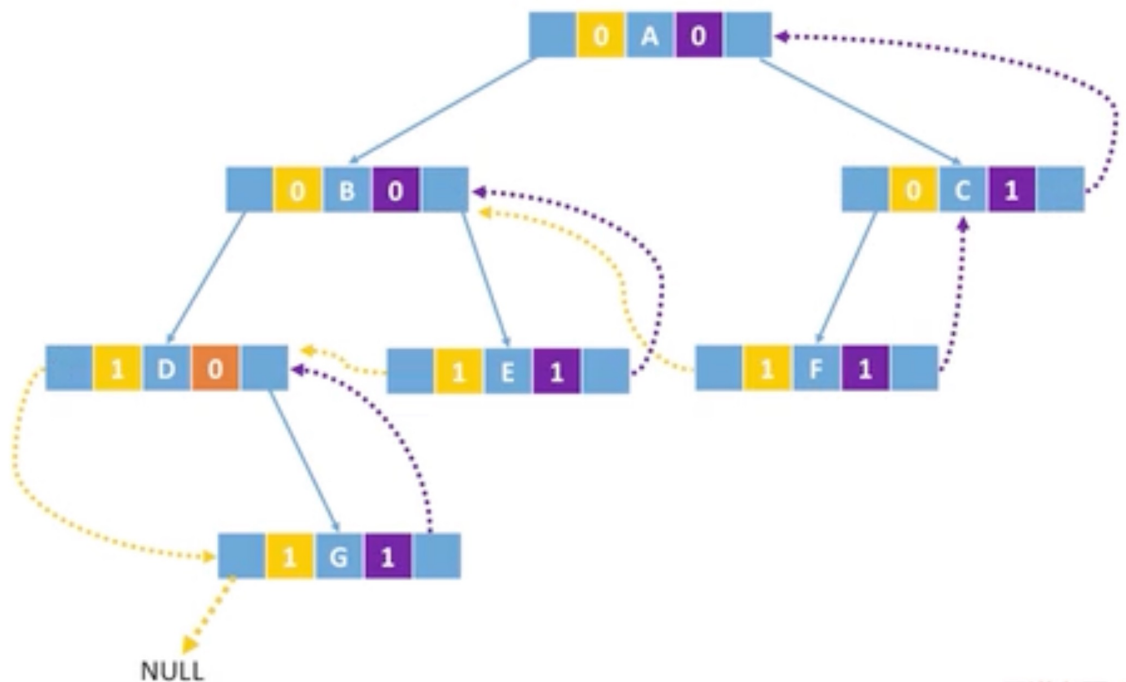
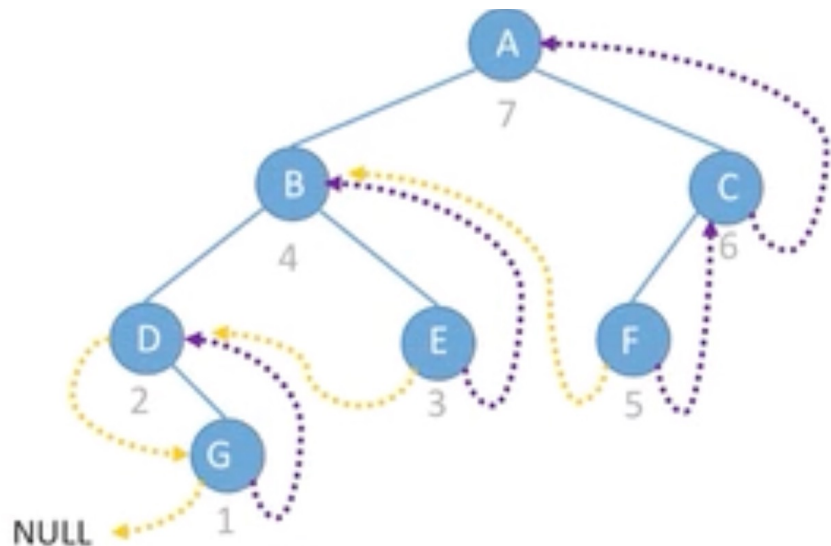
前序线索二叉树



前序遍历: ABDGECF



后序线索二叉树



后序遍历: GDEBFCA



作用：方便从一个指定结点出发，找到其前驱、后继；方便遍历

存储结构

在普通二叉树结点的基础上，增加两个标志位 **ltag** 和 **rtag**

$ltag == 1$ 时，表示 lchild 指向前驱； $ltag == 0$ 时，表示 lchild 指向左孩子

$rtag == 1$ 时，表示 rchild 指向后继； $rtag == 0$ 时，表示 rchild 指向右孩子

三种线索二叉树

中序线索二叉树 以中序遍历序列为依据进行“线索化”

先序线索二叉树 以先序遍历序列为依据进行“线索化”

后序线索二叉树 以后序遍历序列为依据进行“线索化”

几个概念

线索 指向前驱/后继的指针称为线索

中序前驱/中序后继；先序前驱/先序后继；后序前驱/后序后继

手算画出线索二叉树

① 确定线索二叉树类型——中序、先序、or 后序？

② 按照对应遍历规则，确定各个结点的访问顺序，并写上编号

③ 将 $n+1$ 个空链域连上前驱、后继

线索二叉树



5.5.2 中序线索二叉树的建立和遍历



类定义

```
template <class T>
struct ThreadNode {           //线索二叉树的结点类
    int ltag, rtag;           //线索标志
    ThreadNode<T> *leftChild, *rightChild;
                                //线索或子女指针
    T data;                   //结点数据
    ThreadNode ( const T item) //构造函数
        : data(item), leftChild (NULL),
          rightChild (NULL), ltag(0), rtag(0) {}
};
```



```
template <class T>
```

```
class ThreadTree {
```

//线索化二叉树类

```
protected:
```

```
    ThreadNode<T> *root;    //树的根指针
```

```
    void createInThread (ThreadNode<T> *current,  
        ThreadNode<T> *& pre);
```

//中序遍历建立线索二叉树

```
    ThreadNode<T> *parent (ThreadNode<T> *t);
```

//寻找结点t的双亲结点



public:

```
ThreadTree () : root (NULL) { } //构造函数  
void createInThread(); //建立中序线索二叉树  
ThreadNode<T> *First (ThreadNode<T> *current);  
//寻找中序下第一个结点  
ThreadNode<T> *Last (ThreadNode<T> *current);  
//寻找中序下最后一个结点  
ThreadNode<T> *Next (ThreadNode<T> *current);  
//寻找结点在中序下的后继结点  
ThreadNode<T> *Prior (ThreadNode<T> *current);  
//寻找结点在中序下的前驱结点
```

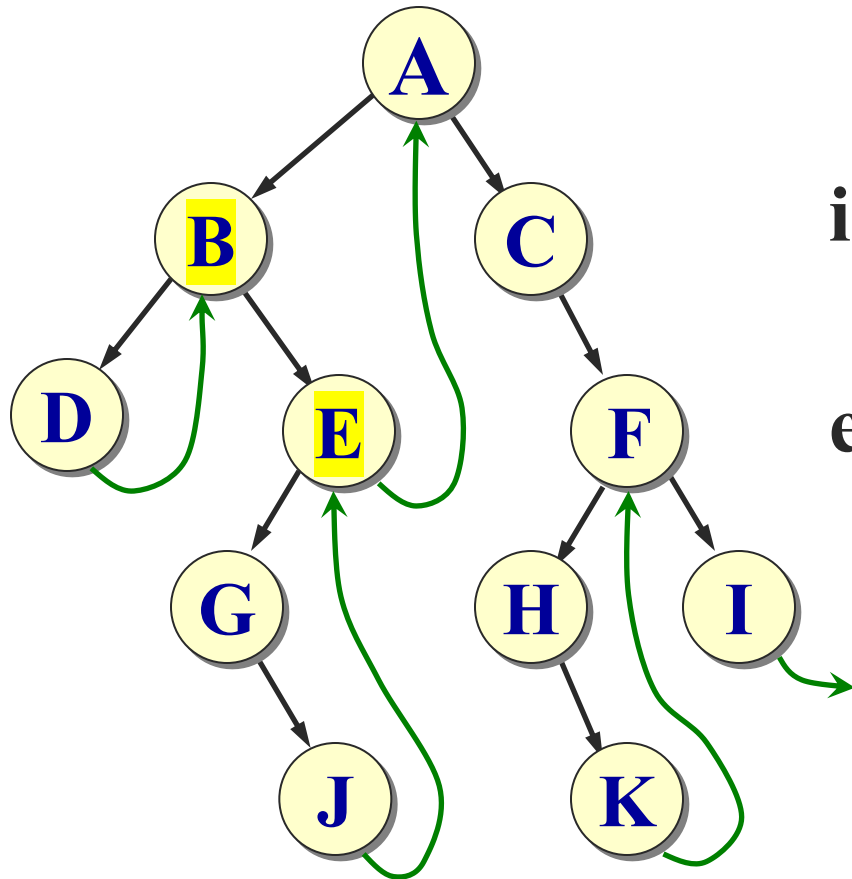
.....

程序5.22

```
};
```



寻找当前结点在中序下的后继

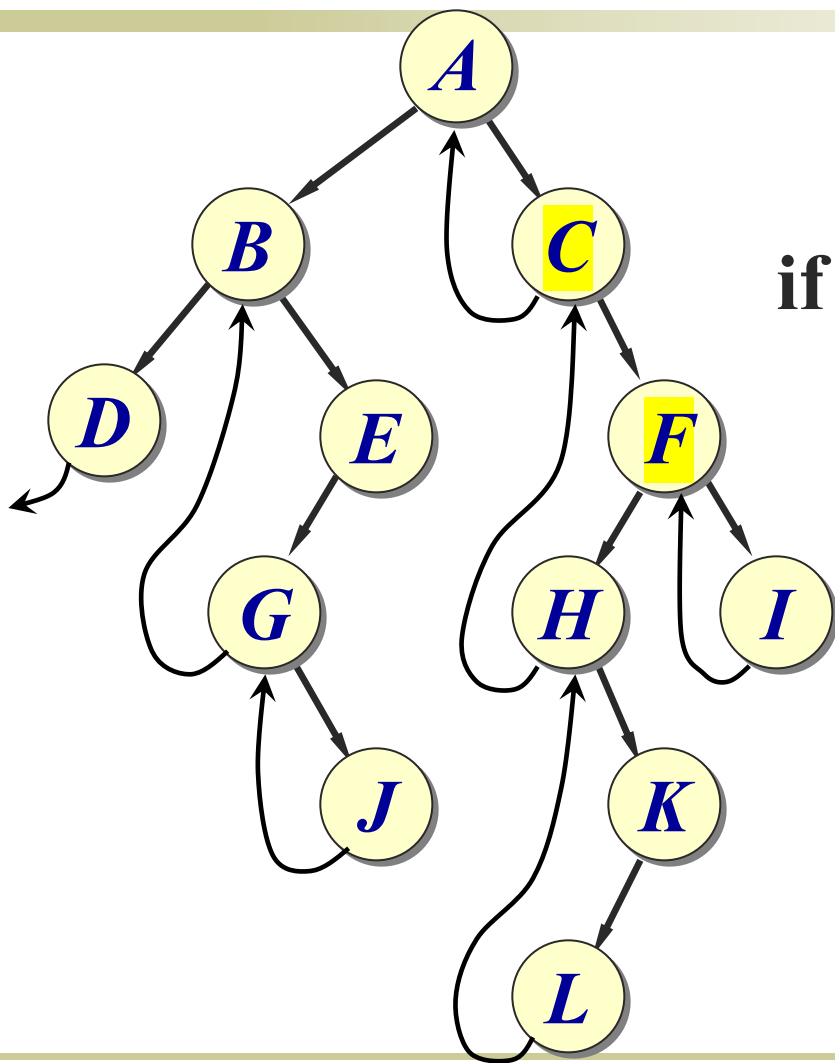


if (current->rtag == 1)
后继为 current->rightChild
else //current->rtag == 0
后继为当前结点右子树
的中序下的第一个结点

中序: DBGJEACHKFI



寻找当前结点在中序下的前驱



if (current->ltag == 1)

前驱为current->leftChild

else //current->ltag == 0

前驱为当前结点左子树
中序下的最后一个结点

中序: DBGJEACHLKFI



通过中序遍历建立中序线索化二叉树



```
template <class T>
void ThreadTree<T>::createInThread ( ) {
    ThreadTree<T> *pre=NULL; //前驱结点指针
    if (root != NULL) {      //非空二叉树，线索化
        createInThread (root, pre); //中序遍历线索化二叉树
        pre->rightChild = pre; pre->rtag = 1;
                                   //后处理中序最后一个结点
    }
};
```



```
template <class T>
void ThreadTree<T>::createInThread (ThreadNode<T>
    *current, ThreadNode<T> *& pre) {
    //通过中序遍历, 对二叉树进行线索化
    if (current == NULL) return;
    createInThread (current->leftChild, pre);
    //递归, 左子树线索化

    if (current->leftChild == NULL) {
        //建立当前结点的前驱线索
        current->leftChild = pre;  current->ltag = 1;
    }
}
```



```
if (pre != NULL && pre->rightChild == NULL)
```

```
    //建立前驱结点的后继线索
```

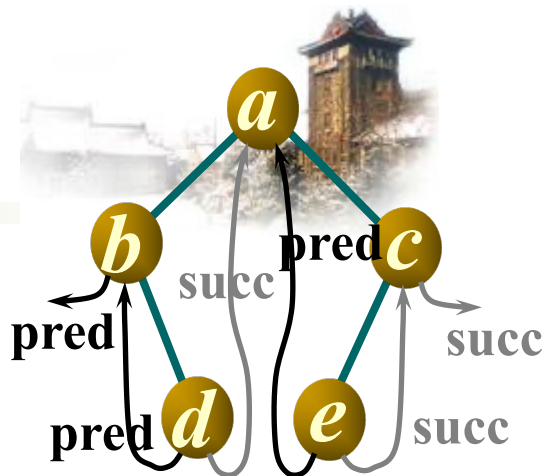
```
    { pre->rightChild = current; pre->rtag = 1; }
```

```
pre = current;    //前驱跟上, 当前指针向前遍历
```

```
createInThread (current->rightChild, pre);
```

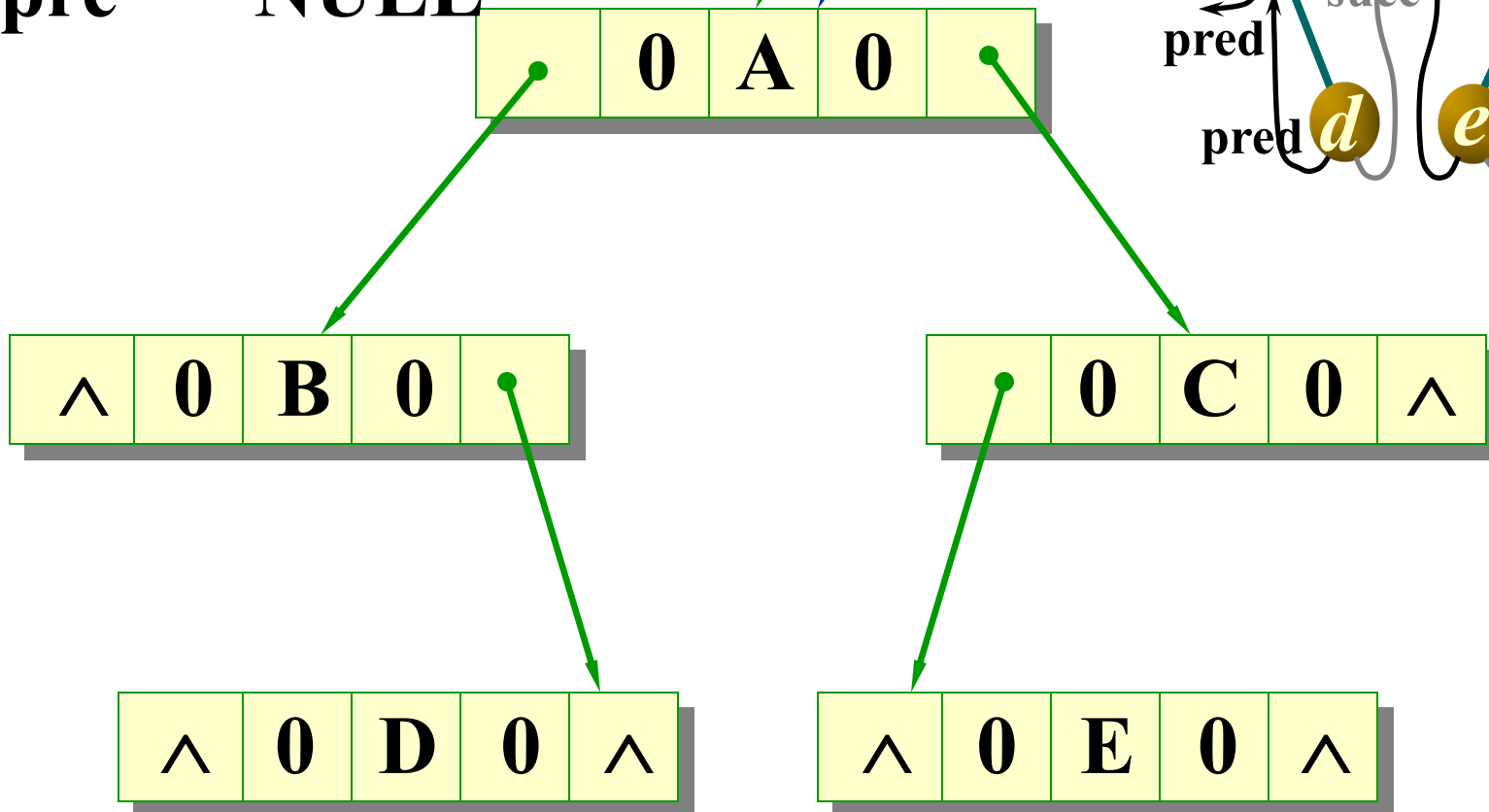
```
    //递归, 右子树线索化
```

```
};
```

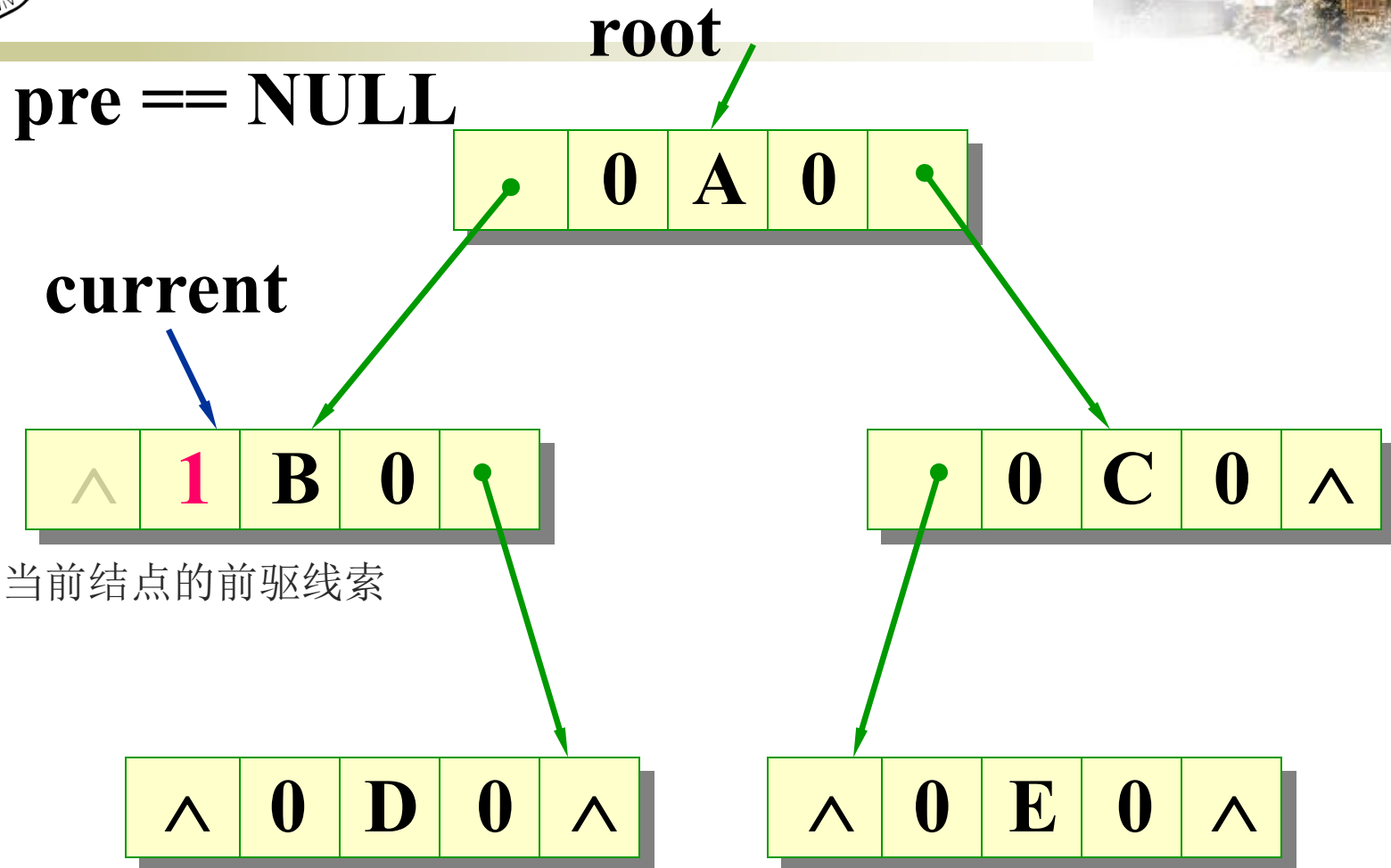


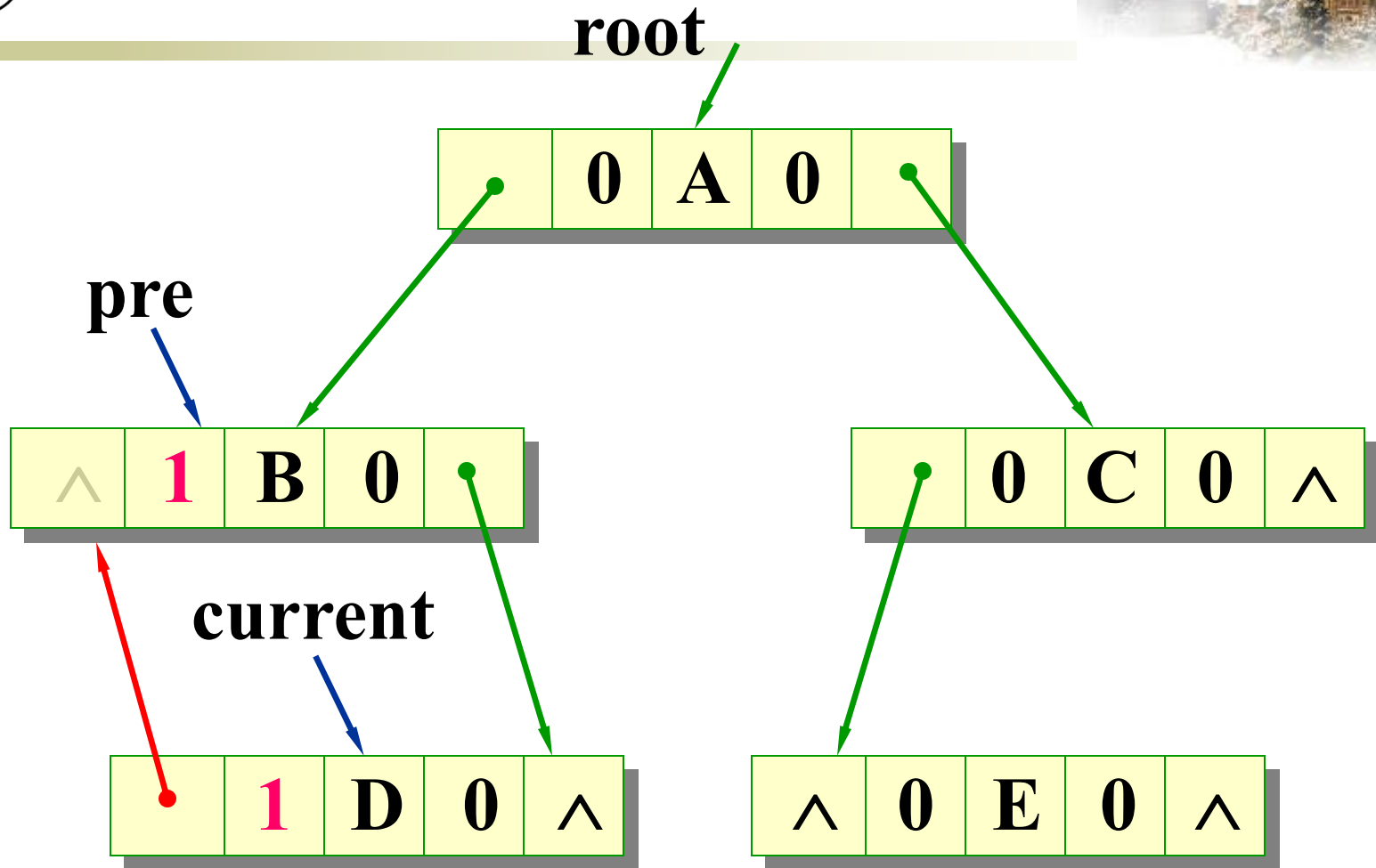
pre == NULL

root current



根据程序5.24创建中序线索二叉树

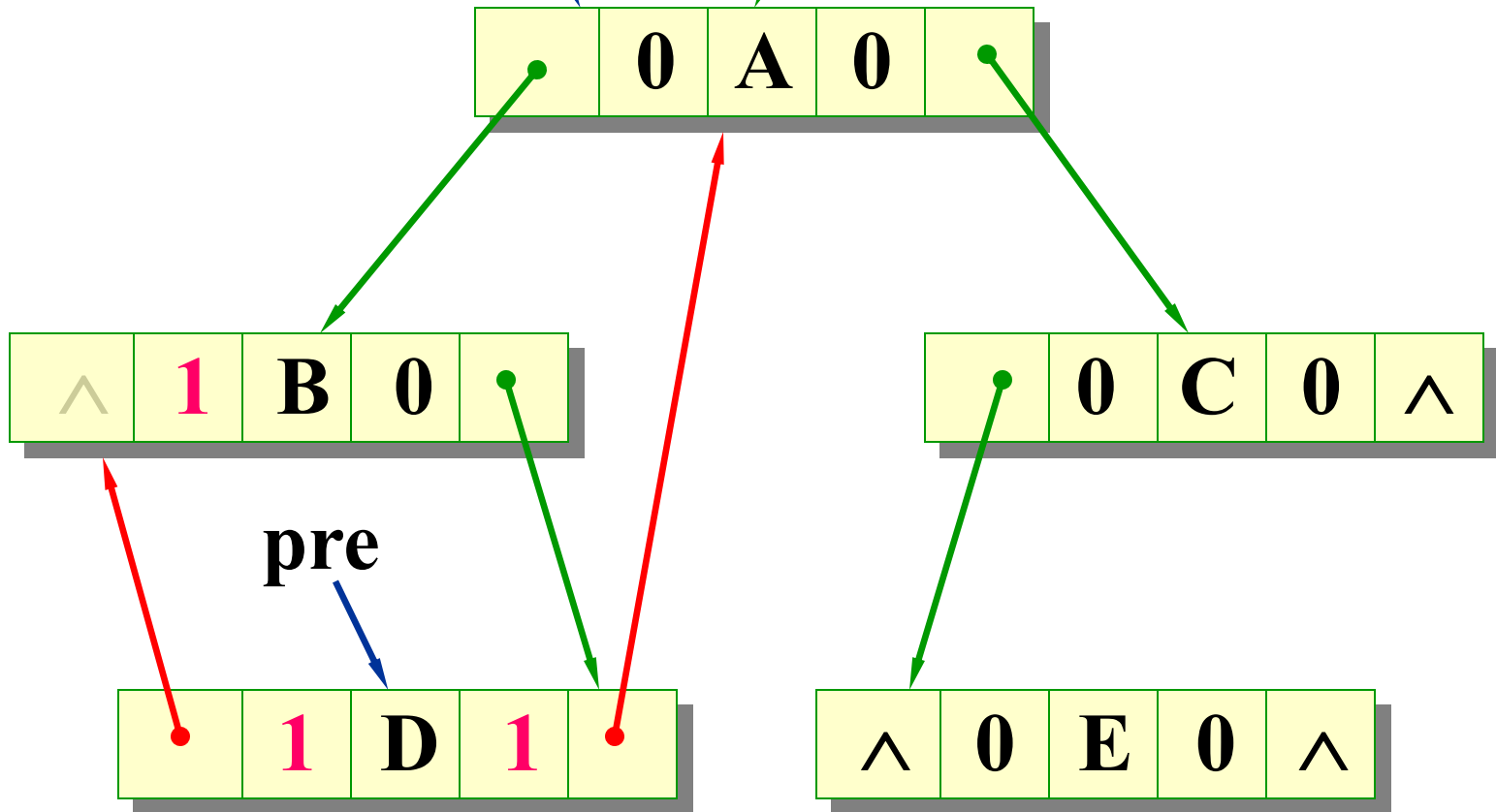




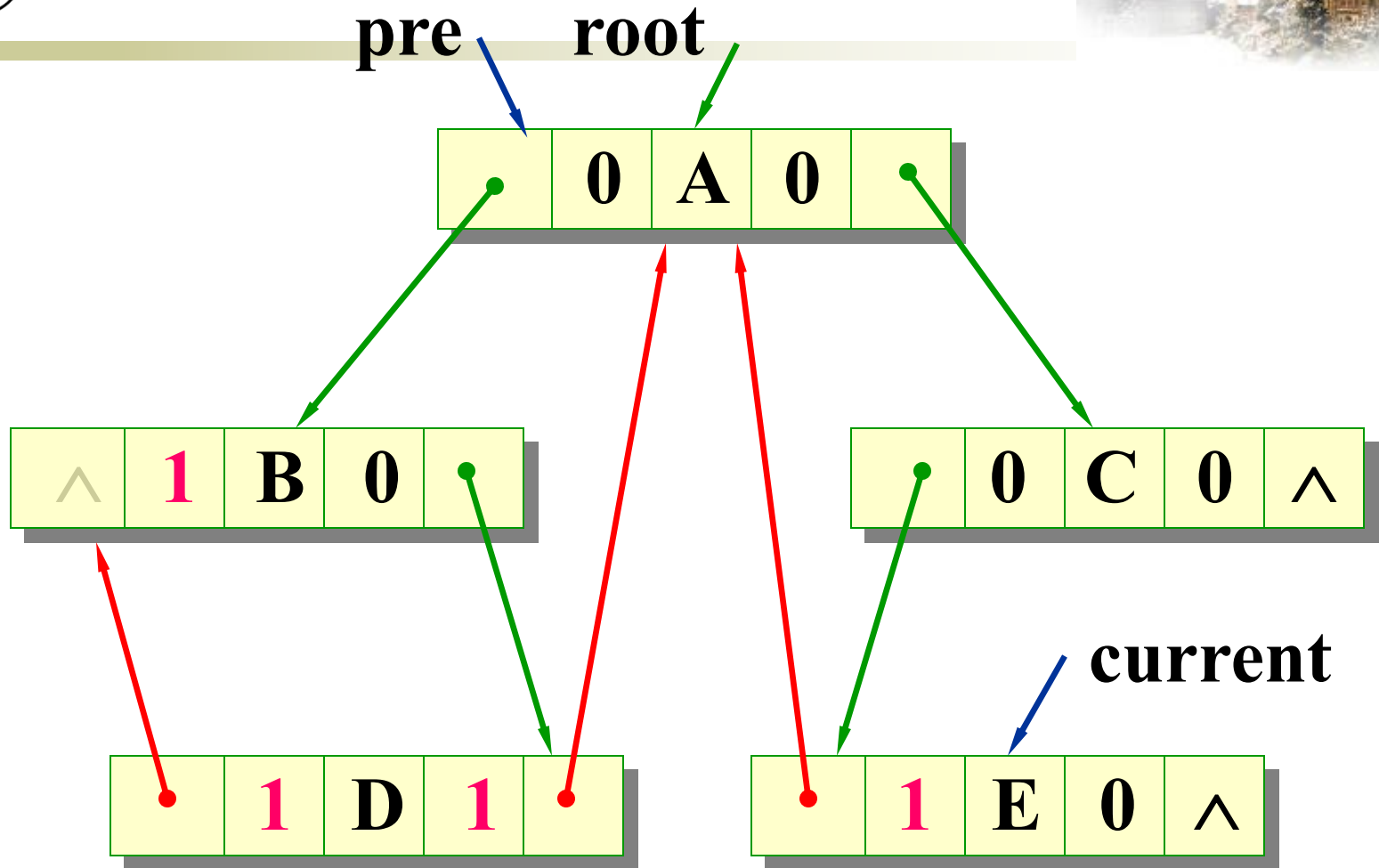
建立当前结点的前驱线索



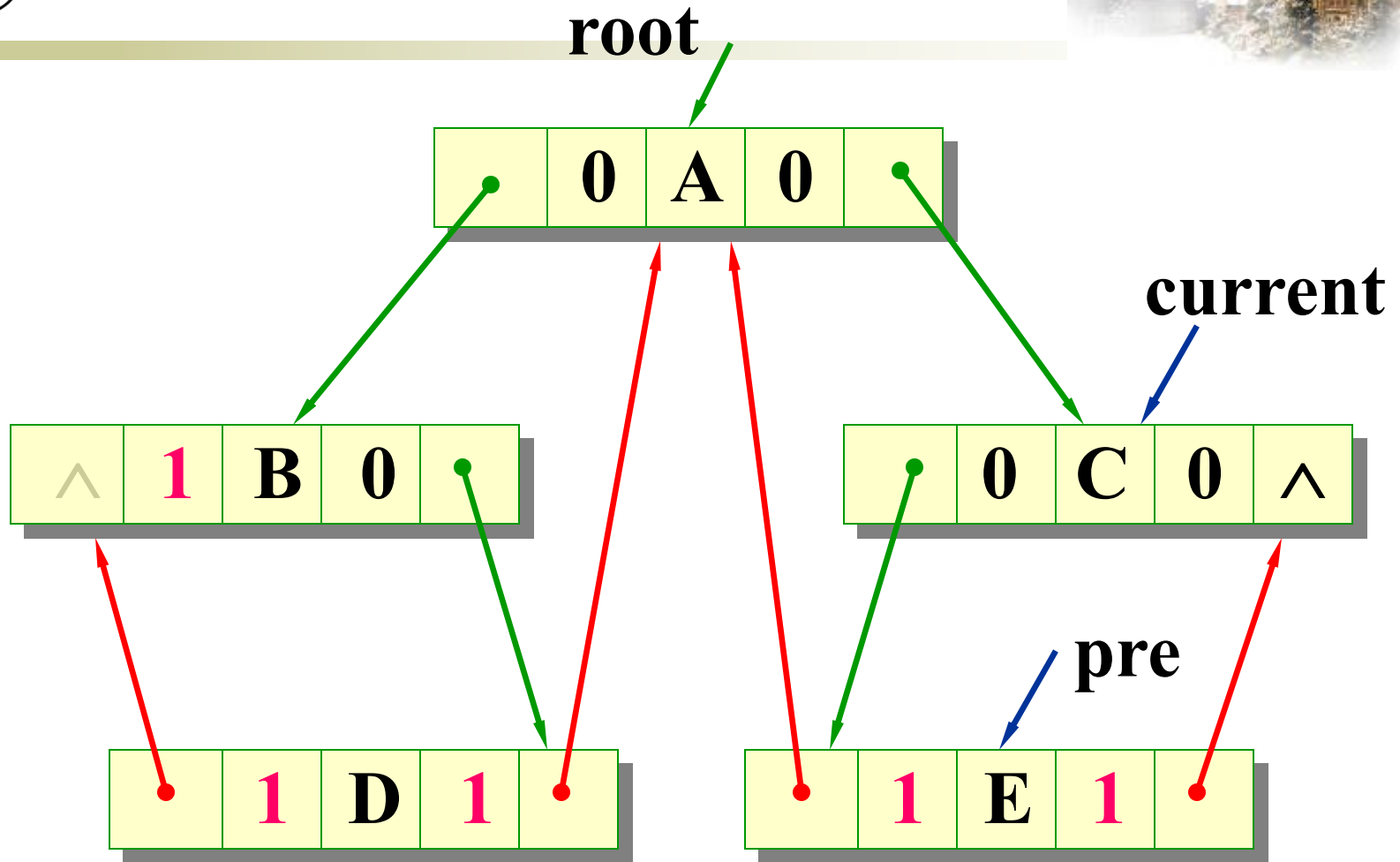
current **root**



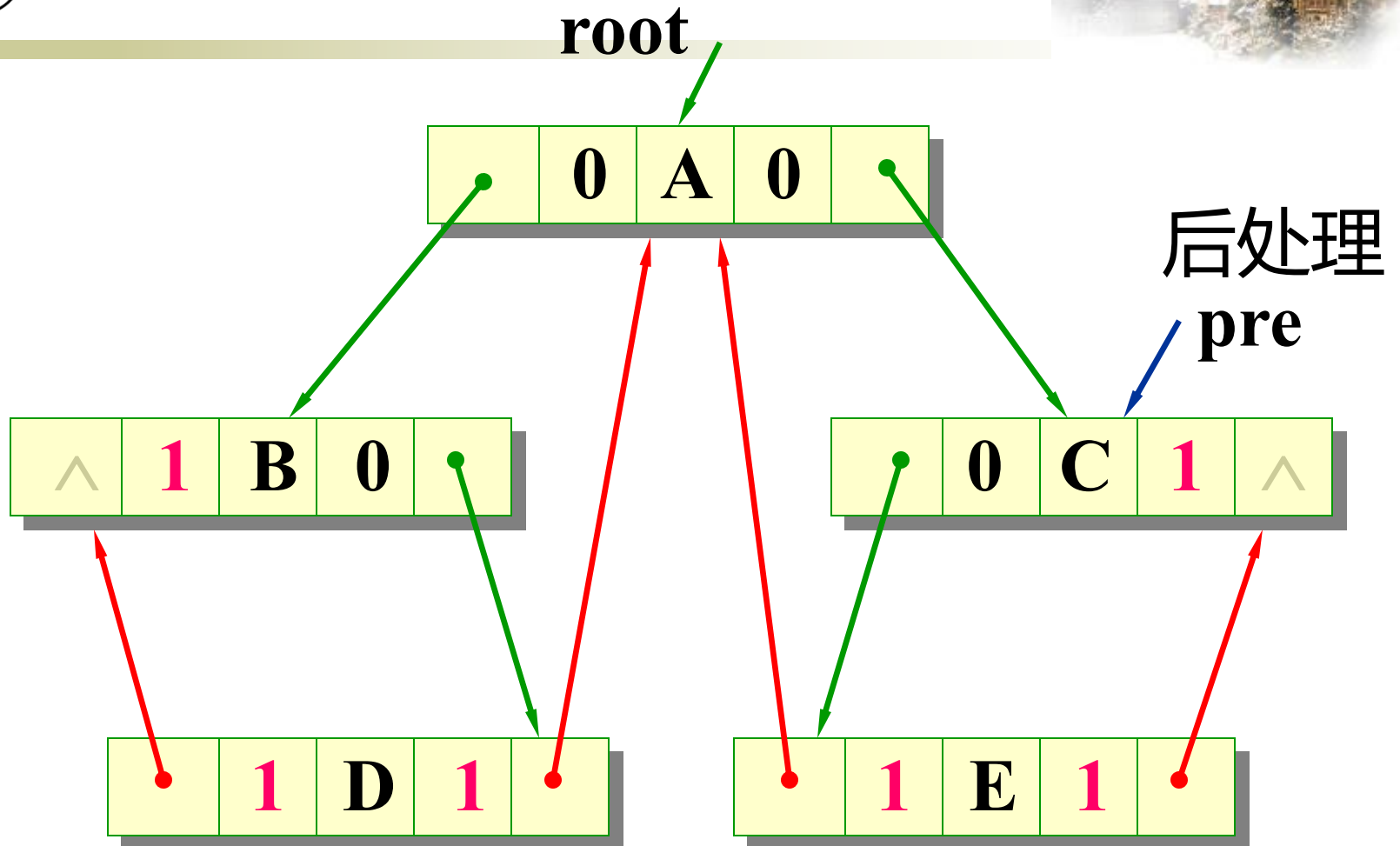
建立当前结点的后继线索



建立当前结点的前驱线索

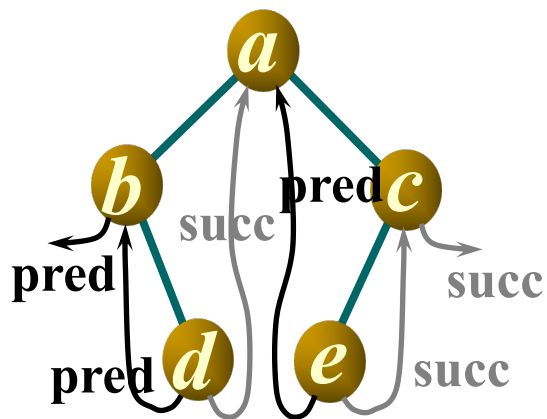


建立当前结点的后继线索





在中序线索二叉树实现前序遍历 程序5.25



前序遍历序列: abdce

后序遍历序列: dbeca

在中序线索二叉树实现后序遍历 程序5.26



- 判断线索二叉树中*p结点有右孩子结点的条件是
()
 - A. $p \neq \text{NULL}$ B. $p \rightarrow \text{rchild} \neq \text{NULL}$
 - C. $p \rightarrow \text{rtag} == 0$ D. $p \rightarrow \text{rtag} == 1$
- 若X是二叉中序线索树中一个有左孩子的结点，且X不为根，则X的前驱为 ()
 - A. X的双亲 B. X的右子树中最左的结点
 - C. X的左子树中最右的结点 D. X的左子树中最右的叶结点



通过前序遍历建立前序线索化二叉树

注意要在左孩子不是前驱线索的情况下进行

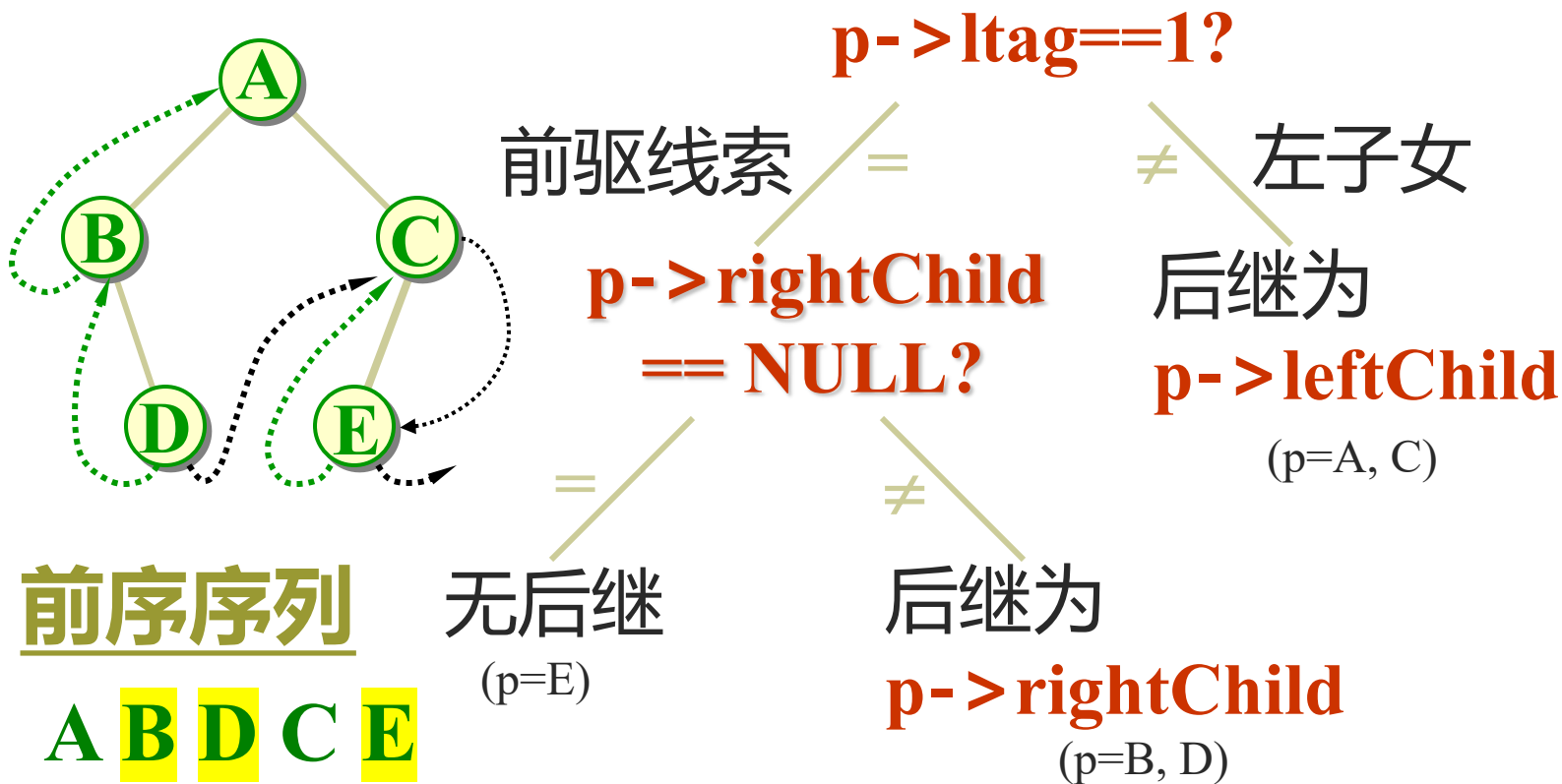
通过后序遍历建立后序线索化二叉树



5.5.4 前序后序线索化二叉树



- 在前序线索化二叉树中寻找当前结点的后继





5.5.4 前序后序线索化二叉树



- 在前序线索化二叉树中寻找当前结点的后继

$p \rightarrow rtag == 1?$

后继线索 $\begin{matrix} / \\ = \end{matrix}$ $\begin{matrix} \backslash \\ \neq \end{matrix}$ 右子女

$p \rightarrow rightChild$ $p \rightarrow leftChild == NULL?$
($p=C, D$)

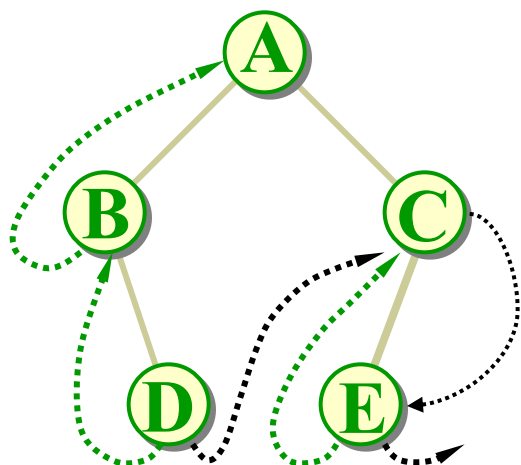
$\begin{matrix} / \\ = \end{matrix}$ $\begin{matrix} \backslash \\ \neq \end{matrix}$
 $p \rightarrow rightChild == NULL?$ $p \rightarrow leftChild$
($p=A$)

$\begin{matrix} / \\ = \end{matrix}$ $\begin{matrix} \backslash \\ \neq \end{matrix}$
 $p \rightarrow rightChild == NULL?$ $p \rightarrow rightChild$
($p=B$)

$q = p \rightarrow parent$
 $q \rightarrow rightChild == p?$

$q.parent \rightarrow rightChild$

$q \rightarrow rightChild$



前序序列

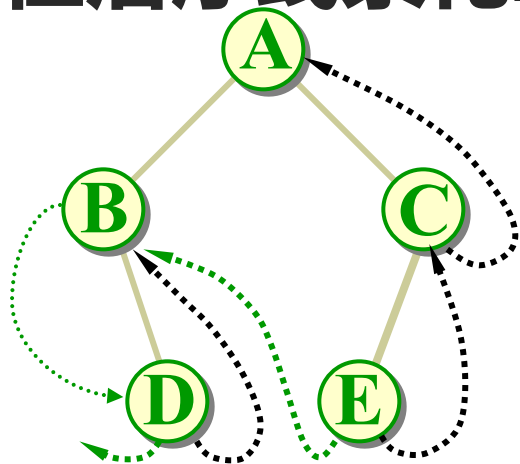
A B **D** **C** E



5.5.4 前序后序线索化二叉树

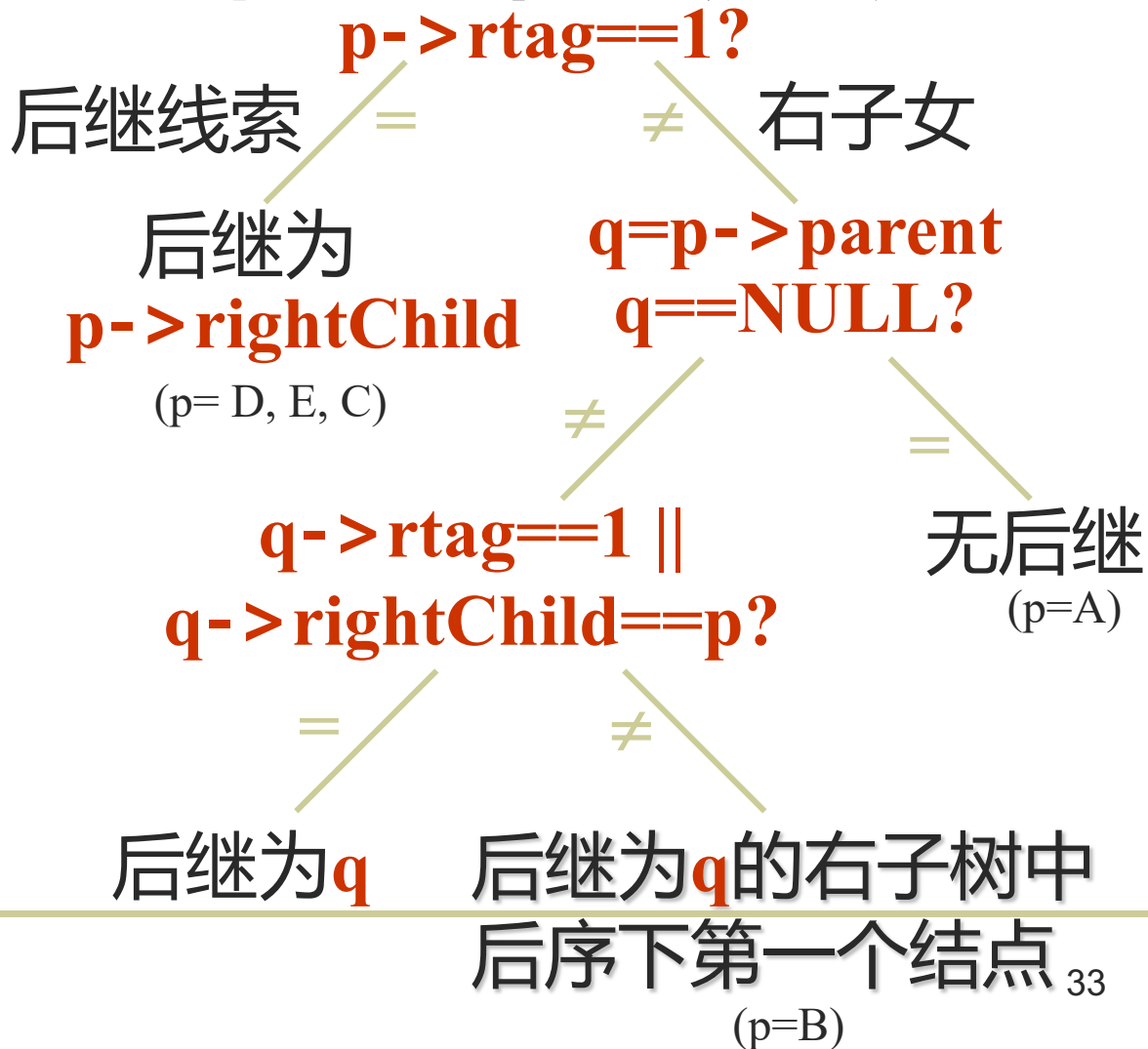


在后序线索化二叉树中寻找当前结点的后继



后序序列

D **B** **E** **C** A



求p的后继